

Instructions for Authors of SBC Conferences

Papers and Abstracts

Luciana P. Nedel ^{*1}, Rafael H. Bordini ^{†2}, Flávio Rech Wagner ^{‡1}, Jomi F. Hübner ^{§3}

¹Instituto de Informática – Universidade Federal do Rio Grande do Sul (UFRGS)
Caixa Postal 15.064 – 91.501-970 – Porto Alegre – RS – Brazil

²Department of Computer Science – University of Durham
Durham, U.K.

³Departamento de Sistemas e Computação
Universidade Regional de Blumenau (FURB) – Blumenau, SC – Brazil

{nedel, flavio}@inf.ufrgs.br, R.Bordini@durham.ac.uk, jomi@inf.furb.br

Abstract. *This meta-paper describes the style to be used in articles and short papers for SBC conferences. For papers in English, you should add just an abstract while for the papers in Portuguese, we also ask for an abstract in Portuguese (“resumo”). In both cases, abstracts should not have more than 10 lines and must be in the first page of the paper.*

Resumo. *Este meta-artigo descreve o estilo a ser usado na confecção de artigos e resumos de artigos para publicação nos anais das conferências organizadas pela SBC. É solicitada a escrita de resumo e abstract apenas para os artigos escritos em português. Artigos em inglês deverão apresentar apenas abstract. Nos dois casos, o autor deve tomar cuidado para que o resumo (e o abstract) não ultrapassem 10 linhas cada, sendo que ambos devem estar na primeira página do artigo.*

1. General Information

All full papers and posters (short papers) submitted to some SBC conference, including any supporting documents, should be written in English or in Portuguese. The format paper should be A4 with single column, 3.5 cm for upper margin, 2.5 cm for bottom margin and 3.0 cm for lateral margins, without headers or footers. The main font must be Times, 12 point nominal size, with 6 points of space before each paragraph. Page numbers must be suppressed.

Full papers must respect the page limits defined by the conference. Conferences that publish just abstracts ask for **one**-page texts.

*Info about your graduation, post-graduation, master course, PhD, etc. HERE

†Informação sobre Graduações, Pós-graduações, Mestrados e etc., incluindo as instituições AQUI

‡Informação sobre Graduações, Pós-graduações, Mestrados e etc., incluindo as instituições AQUI

§Informação sobre Graduações, Pós-graduações, Mestrados e etc., incluindo as instituições AQUI

2. Fundamentação Teórica

2.1. Computação Paralela

Do período de 1986 até 2002, o desempenho dos processadores aumentava cerca de 52% a cada ano (?). No entanto, desde 2002 o crescimento de desempenho das novas gerações de microprocessadores tem desacelerado cada vez mais, isso se deve ao limite físico do silício (?). Desse modo, por volta de 2005, a grande maioria das fabricantes de microprocessadores decidiu que o caminho para o ganho de desempenho estava direcionado à programação paralela (?).

(?) aponta que na computação paralela "um programa é aquele no qual múltiplas tarefas cooperam intimamente para resolver um problema". Nesse sentido, operações matriciais representam um problema adequado para o paralelismo, já que o cálculo de elementos diferentes podem ser distribuídos em diferentes unidades de execução.

2.2. Multiplicação de Matrizes

De acordo com ?, o problema clássico para multiplicação de matrizes envolve duas matrizes dadas, A e B, e C uma nova matriz a ser calculada. A operação que define cada elemento C_{ij} pode ser dada como:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj} \quad (1)$$

Ou seja, cada elemento C_{ij} é obtido pelo produto escalar entre a linha i de A e a coluna j de B. Dessa forma, o algoritmo sequencial possui um custo $O(n^3)$ para matrizes de dimensão N por N.

2.3. Princípio da localidade espacial

? mostram que segundo o princípio da localidade espacial, quando um dado é acessado, existe uma grande possibilidade dos dados armazenados próximos serem necessários. Dessa forma, quando um dado não é encontrado em *cache*, o processador busca um bloco de dados próximos ao dado na memória principal.

Nesse sentido, segundo ?, em C/C++ as matrizes multidimensionais são armazenadas em ordem *row-major*, ou seja, os elementos de uma mesma linha ocupam posições consecutivas na memória. Por conseguinte, percorrer os elementos de uma linha de forma consecutiva, tende a favorecer a localidade espacial, já percorrer por coluna desfavorece o aproveitamento de dados em *cache*.

2.4. Transposição de matrizes

No cenário de multiplicação de matrizes o acesso à memória da matriz B terá um custo maior que A. Assim, a transposição da matriz, definido por ? como a permutação dos elementos de cada i -ésima linha, pela j -ésima coluna, pode resolver o problema do armazenamento na memória, alterando apenas a forma de cálculo da matriz C para manter o resultado, computando com os elementos da linha " j " ao invés da coluna.

2.5. Algoritmo de Strassen para multiplicação de matrizes

Segundo ? o algoritmo de Strassen para multiplicação de matrizes foi uma descoberta revolucionária na álgebra linear computacional. A ideia central desse algoritmo é multiplicar matrizes de tamanho $2N \times 2N$ com apenas 7 operações, utilizando a estratégia de divisão e conquista aplicar a matrizes $N \times N$. O algoritmo surge da decomposição da forma:

$$XY = \sum_{k=1}^7 u_k(X)v_k(Y)W_k \quad (2)$$

Em que u_k e v_k são formas lineares escolhidas no espaço das matrizes $2N \times 2N$ e W_k são sete matrizes de tamanho $2N \times 2N$. Aplicando esse algoritmo é possível reduzir o custo computacional para $O(n^{\log_2 7})$ operações aritméticas ?.

2.6. Métricas de Desempenho

Entre as principais métricas para avaliação do desempenho de algoritmos destacam-se: Tempo de execução; *Overhead* paralelo total; *Speedup*; Eficiência;

? definem o tempo de execução de um algoritmo sequencial T_s como o tempo decorrido entre o início e o fim de sua execução em um computador sequencial. Já o tempo de execução de um algoritmo paralelo T_p é definido como o tempo decorrido a partir do momento em que a computação paralela se inicia até o momento da execução do último elemento de processamento.

Os custos computacionais adicionados devido à paralelização são encapsulados em uma expressão chamada *overhead*, que pode ser definida como o tempo total gasto por todos os elementos coletivamente, além do tempo necessário pelo algoritmo sequencial ?. Dessa forma, sendo p o número de *threads* *overhead* paralelo total pode ser calculado como:

$$T_o = pT_p - T_s \quad (3)$$

Speedup é a medida que mostra o benefício relativo de programar paralelamente em relação ao programa sequencial ?. Assim, o *Speedup* do sistema pode ser calculado como a razão do tempo de execução sequencial pelo tempo em paralelo como na equação abaixo:

$$S_p = \frac{T_s}{T_p} \quad (4)$$

Já a eficiência é uma medida da fração de tempo na qual um elemento de processamento está utilmente empregado ?. A eficiência pode ser calculada como:

$$E_p = \frac{T_s}{pT_p}. \quad (5)$$

3. Trabalhos Relacionados

3.1. Parallel matrix transpose algorithms on distributed memory concurrent computers

O artigo “Parallel matrix transpose algorithms on distributed memory concurrent computers” ? trata da criação de um algoritmo referente à realização de operações de transposição de matrizes utilizando-se de computação paralela.

Para realizar o procedimento, o algoritmo separa os processadores disponíveis em uma matriz de duas dimensões e, utilizando distribuição de dados bloco-cíclica (block cyclic data distribution), divide sub-seções da matriz a serem analisadas por processadores específicos.

Dependendo da divisão dos processadores na representação 2D, sendo P o número de linhas e Q o número de colunas, duas situações distintas podem ocorrer: o máximo divisor comum (MDC) entre P e Q é calculado. Caso ele seja igual a 1, os números são matematicamente primos, e todos os processadores precisam realizar os procedimentos realizando ‘comunicação total’ entre si.

Caso o MDC seja maior que 1, o algoritmo divide os blocos de processadores em grupos delimitados pelo MDC, nos quais todas as operações de transposição entre blocos de mesmo grupo podem ocorrer simultaneamente, diminuindo a quantidade total de tempo de execução.

A análise final do algoritmo realizada no artigo demonstra que, em uma multiplicação de matrizes, transpor o resultado da operação utilizando o processo descrito aumenta em pouco o tempo total de execução. Ademais, o algoritmo se destaca, em principal, ao suportar versatilidade quanto a proporções em grades bidimensionais não quadradas (com P diferente de Q), não só mantendo alta eficiência, mas superando algoritmos restritos a arranjos quadrados.

Um ponto negativo observável no algoritmo, no entanto, possui uma descrição breve no artigo em si. Quando P e Q tem MDC igual a 1 e a comunicação total é realizada, a sua implementação é não só mais lenta do que o método com MDC maior, mas também não se utiliza da melhor implementação possível em teoria. Os autores optam por uma implementação não ótima que, dada a quantidade arbitrariamente grande de processadores, seria mais simples de ser utilizada.

3.2. Anatomy of High-Performance Matrix Multiplication

Outro artigo mais diretamente relacionado ao tópico é “Anatomy of High-Performance Matrix Multiplication”?, que discorre diretamente sobre implementações otimizadas de operações de multiplicação de matriz, mais especificamente sobre gerenciar de maneira eficiente a hierarquia de memória disponível para obter alto desempenho de maneira consistente.

Os autores identificam que, nas abordagens mais tradicionais, os algoritmos criados não condizem às capacidades reais do hardware ao mensurar performance, como o número finito de entradas no Translation Look-aside Buffer (TLB) e as latências das memórias cache. O algoritmo proposto, portanto, busca segmentar mais o problema utilizando empacotamento a fim de aproveitar mais a memória no nível local.

Para evitar os gargalos de memória descritos, o sistema proposto reestrutura o algoritmo comum da GEMM (General Matrix Multiplication) em níveis distintos: no mais alto nível, a matriz de entrada é particionada de modo que o painel da matriz A é dimensionado para se ajustar ao cache L2 (ou L3) e ao limite de entradas do TLB, enquanto os blocos da matriz B são empacotados para fluir diretamente pelo cache L1.

Em nível de bloco, os dados lidos atribuídos passam pelo processo de empacotamento, a partir do qual os pacotes são inseridos em buffers temporários contíguos, para

que o acesso a eles possa ser mais rápido e direto no processamento.

No micro-kernel, o processamento é realizado em pequenas submatrizes, maximizando a instrução vetorial e o paralelismo ao nível de instrução. O artigo apresenta variados experimentos que demonstram o aumento de desempenho dado por sua abordagem, e é uma base fundamental para análise referente às operações fundamentais de GEMM, e o algoritmo é utilizado em múltiplas bibliotecas atuais de computação de alta performance (HPC), como OpenBLAS e Intel MKL.

Apesar da alta eficiência do procedimento, um ponto negativo observável é que o processo é bastante elaborado e, portanto, contém overhead significativo. Dado que os procedimentos buscam utilizar o máximo possível de performance, o algoritmo é projetado para uma quantidade grande de processamento necessário e, caso a operação seja simples, as soluções implementadas seriam mais custosas do que ótimas em totalidade.

3.3. Relação com o Projeto Principal

Os artigos descritos, para o propósito deste projeto, apresentam não apenas algoritmos eficientes em suas respectivas áreas de ênfase (transposição e multiplicação de matrizes), mas também demonstram, ao longo de suas análises, diversas estratégias de gerenciamento de recursos do sistema para otimizar processos dependentes de matrizes. Dessa forma, mesmo que o presente trabalho não consista em uma reconstrução direta de nenhum dos métodos, ambos os artigos contêm esquemas de particionamento e análise bastante úteis para o desenvolvimento de nosso projeto.

4. Sections and Paragraphs

Section titles must be in boldface, 13pt, flush left. There should be an extra 12 pt of space before each title. Section numbering is optional. The first paragraph of each section should not be indented, while the first lines of subsequent paragraphs should be indented by 1.27 cm.

4.1. Subsections

The subsection titles must be in boldface, 12pt, flush left.

5. Figures and Captions

Figure and table captions should be centered if less than one line (Figure ??), otherwise justified and indented by 0.8cm on both margins, as shown in Figure ?. The caption font must be Helvetica, 10 point, boldface, with 6 points of space before and after each caption.

In tables, try to avoid the use of colored or shaded backgrounds, and avoid thick, doubled, or unnecessary framing lines. When reporting empirical data, do not use more decimal digits than warranted by their precision and reproducibility. Table caption must be placed before the table (see Table 1) and the font used must also be Helvetica, 10 point, boldface, with 6 points of space before and after each caption.

6. Images

All images and illustrations should be in black-and-white, or gray tones, excepting for the papers that will be electronically available (on CD-ROMs, internet, etc.). The image



Figura 1. A typical figure

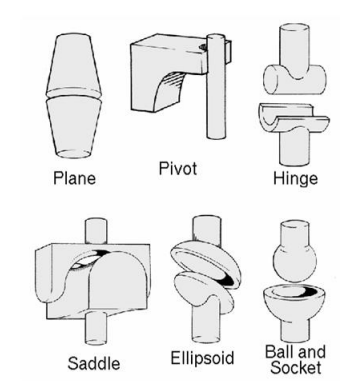


Figura 2. This figure is an example of a figure caption taking more than one line and justified considering margins mentioned in Section ??.

resolution on paper should be about 600 dpi for black-and-white images, and 150-300 dpi for grayscale images. Do not include images with excessive resolution, as they may take hours to print, without any visible difference in the result.

Referências

- Boulic, R. and Renault, O. (1991). 3d hierarchies for animation. In Magnenat-Thalmann, N. and Thalmann, D., editors, *New Trends in Animation and Visualization*. John Wiley & Sons Ltd.
- Knuth, D. E. (1984). *The T_EX Book*. Addison-Wesley, 15th edition.
- Smith, A. and Jones, B. (1999). On the complexity of computing. In Smith-Jones, A. B., editor, *Advances in Computer Science*, pages 555–566. Publishing Press.

Tabela 1. Variables to be considered on the evaluation of interaction techniques

	Chessboard top view	Chessboard perspective view
Selection with side movements	6.02 $\pm$ 5.22	7.01 $\pm$ 6.84
Selection with in- depth movements	6.29 $\pm$ 4.99	12.22 $\pm$ 11.33
Manipulation with side movements	4.66 $\pm$ 4.94	3.47 $\pm$ 2.20
Manipulation with in- depth movements	5.71 $\pm$ 4.55	5.37 $\pm$ 3.28